Programação Vetorial

Modificação de *rank*, ordenação e reduções

Prof. Edson Alves

Faculdade UnB Gama

Sumário

1. **Modificação do *rank* e da profundidade**
2. **Ordenação**
3. **Reduções e varreduras**

Mistura

- A função monádica ↑ (*mix*, mistura) permite a criação de *arrays* de *rank* maior
- Ela rearranja os elementos de um *array* em um novo *array* de *rank* uma unidade maior, com um nível a menos de aninhamento

```
    ↑(2 3 5) (7 9 11)
2 3  5
7 9 11
```

- Se os elementos tem formas distintas, serão utilizados elementos para preencher os espaços em branco, de acordo com o tipo do primeiro componente do elemento
- Em geral, inteiros são preenchidos com zero e caracteres com o espaço em branco

```
    ↑(2 3) 5 (7 9 11) ⋄ ↑'string' 'abc'
2 3  0
5 0  0
7 9 11
string
abc
```

Novo símbolo

Símbolo	Aridade	Descrição
↑ (*mix*)	monádico	Rearranja os elementos do *array* em um novo *array* com *rank* uma unidade maior

Unicode	TAB	APL
U+2191	^ | <tab>	APL + y

Cisão

- A função monádica ↓ (*split*, cisão) gera, a partir do argumento, um novo *array* aninhado, com *rank* uma unidade menor do que o argumento
- Caso o argumento seja escalar, o retorno terá *rank* zero
- Se o argumento tem forma $d_1\ d_2\ \ldots\ d_N$, então o retorno terá forma $d_1\ d_2\ \ldots\ d_{N-1}$, cujos elemento de profundidade 1 tem forma d_N

```
    A ← 2 3 ⍴ 2 3 5 7 9 11
    ⍴ A
2 3
    ↓A
┌─┬─────┐
│2│3 5 7│
├─┼─────┤
│3│9 11 │
└─┴─────┘
    ⍴ ↓A
2
    ⍴⍴ A ⋄ ⍴⍴ ↓A
2
1
```

Novo símbolo

Símbolo	Aridade	Descrição
↓ (*split*)	monádico	Rearranja os elementos do *array* em um novo *array* com *rank* uma unidade menor

Unicode	TAB	APL
U+2193	v | <tab>	APL + u

Vetores inclusos

- Aplicar a cisão em um vetor **v** (*array* de dimensão 1) resultará em um *array* de dimensão zero cujo conteúdo é próprio **v**

```
    ↓ 2 3 5 7 11
2 3 5 7 11
    ⍴ ↓ 2 3 5 7 11

    ⍴⍴ ↓ 2 3 5 7 11
0
```

- Neste caso, o retorno de ↓**v** tem a mesma forma do vetor ~~0~~
- Este retorno é denominado vetor incluso (*enclosed vector*) ou escalar aninhado (*nested scalar*)

Escalares aninhados

- Aplicar uma cisão em um escalar simples (profundidade zero) produz o próprio escalar

```
      ⍴⍴ ↓ 2 ⋄ ≡ ↓ 2
0
0
```

- Aplicar uma cisão em um escalar aninhado aumenta sua profundidade em uma unidade

```
      ≡ 2 3 5 7 11 ⋄ ≡ ↓↓ 2 3 5 7 11
1
3
```

- Aplicar uma mistura em um escalar aninhado reduz sua profundidade em uma unidade

```
      ≡ ↑↓↓ 2 3 5 7 11
2
```

Inclusão

- A função monádica ⊂ (*enclose*) gera um escalar a partir de qualquer *array*, incluindo-o em um vetor aninhado (escalar incluso)

```
    ⊂ 2 2 ⍴ 2 3 5 7
2 3
5 7
    ⍴⍴ ⊂ 2 2 ⍴ 2 3 5 7
0
```

- A inclusão é usada principalmente em funções escalares, ou que tratam os escalares de forma diferenciada

```
    2 3 5 + 7 11 13
9 14 18
      2 3 5 + ⊂ 7 11 13
9 13 15   10 14 16   12 16 18
```

Novo símbolo

Símbolo	Aridade	Descrição
⊂ (*enclose*)	monádico	Retorna um escalar incluso que contém o *array* passado como parâmetro

Unicode	TAB	APL
U+2282	c c <tab>	APL + z

Extração

- A função monádica ⊃ (*disclose, first*) faz o trabalho inverso da inclusão
- Quando aplicada em um vetor aninhado, ela desfaz a inclusão e retorna o vetor

```
    ⊃⊂ 2 3 5 7
2 3 5 7
    ⍴⍴ ⊃⊂ 2 3 5 7
1
```

- Se for aplicada em um *array*, ela extrai o primeiro elemento do nível de profundidade 1

```
    ⊃ 2 3 5 7
2
    ⊃ ((2 3 5) 7 ((11 13) (17 19 23)))
2 3 5
```

Novo símbolo

Símbolo	Aridade	Descrição
⊃ (*disclose*, *first*)	monádico	Retorna o primeiro elemento no nível de profundidade 1

Unicode	TAB	APL
U+2283	)) <tab>	APL + x

Ranqueamento ascendente

- APL não disponibiliza uma primitiva para a ordenação dos elementos de um *array*
- Para ordenar um vetor é preciso recorrer as funções de ranqueamento
- A função monádica ⍋ (*grade up*) ranqueia um vetor ascendentemente
- Ela retorna um vetor de índices, cujo i-ésimo elemento indica o índice do i-ésimo menor elemento do argumento

```
      a ← 6 9 3 1 4 7 1 8 0          ⍝ dígitos da expansão decimal de ln 2
        ⍋a
9 4 7 3 5 1 6 8 2
```

- A ordenação de um vetor pode ser obtida por meio da expressão `a[⍋a]`

```
      a[⍋a]
0 1 1 3 4 6 7 8 9
```

Novo símbolo

Símbolo	Aridade	Descrição
 (*grade up*)	monádico	Ranqueia o argumento de forma ascendente

Unicode	TAB	APL
`U+234B`	`A | <tab>`	`APL + Shift + 4`

Novo símbolo

Símbolo	Aridade	Descrição
`[ ]` (*square brackets*)	-	`v[u]` retorna os elementos do vetor `v` que ocupam os índices indicado pelo vetor `u`

Unicode	TAB	APL
U+005[BD]	-	-

Ordenação lexicográfica

- A ordenação apresentada ordena os elementos de um *array* segundo a ordem lexicográfica

```
A ← 5 3 ⍴ a
      A
6 9 3
1 4 7
1 8 0
6 9 3
1 4 7
      ⍋A
2 5 3 1 4          ⍝ Ordem lexicográfica das linhas
```

- A função diádica ⌷ (*index*) permite, em conjunto com uma inclusão, ordenar um *array* de forma arbitrária

```
sortA ← {(⊂⍋⍵)⌷⍵}        ⍝ Ordenação ascendente (dfn)
sortB ← (⊂⍋)⌷⊢           ⍝ Ordenação ascendente (trem)
```

Novo símbolo

Símbolo	Aridade	Descrição
⌷ (*index*)	diádico	Retorna os índices do argumento à direita indicados pelo escalar à esquerda

Unicode	TAB	APL
U+2337	[| <tab>	APL + Shift + l

Ranqueamento descendente

- A função monádica ⍒ (*grade down*) retorna o ranqueamento descendente de seu argumento

```
    a ← 6 9 3 1 4 7  1 8 0
    ⍒a
2 8 6 1 5 3 4 7 9
```

- Strings são ordenadas de acordo com os valores dos caracteres na tabela Unicode

```
    sortDesc ← (⊂⍒)⌷⊢
    sortDesc 'abacaxi' 'abobora' 'abacate'
abobora abacaxi abacate
```

Novo símbolo

Símbolo	Aridade	Descrição
⍒ (*grade down*)	monádico	Ranqueia o argumento de forma descendente

Unicode	TAB	APL
U+2352	V | <tab>	APL + Shift + 3

Aplicações do ranqueamento

- É possível obter o índice do menor ou do maior elemento de um *array* usando os *atops* ⊃⍋ e ⊃⍒, respectivamente

```
    a ← 6 9 3 1 4 7 1 8 0
    ⊃⍋ a
9
    ⊃⍒ a
2
```

- O duplo ranqueamento permite obter as posições que cada elemento do *array* ocupará após a ordenação

```
    ⍋a
9 4 7 3 5 1 6 8 2
    ⍋⍋a
6 9 4 2 5 7 3 8 1
```

Ranqueamentos diádicos

- As funções de ranqueamento também tem formas diádicas
- O parâmetro à esquerda indicará o alfabeto ⍺ que será utilzado como critério de ordenação
- No exemplo a seguir os caracteres ímpares devem anteceder os caracteres pares na ordenação

```
    n ← '693147180'
    o ← '1357902468' ⍋ n
    o
4 7 3 6 2 9 5 1 8
    n[o]
113790468
```

- Caracteres que não forem indicado no alfabeto serão considerados equivalentes, ocupando a posição logo após o último caractere indicado

Novo símbolo

Símbolo	Aridade	Descrição
⍋ (*grade up*)	diádico	Ranqueia o argumento de forma ascendente, de acordo com o alfabeto indicado

Unicode	TAB	APL
U+2352	V | <tab>	APL + Shift + 4

Novo símbolo

Símbolo	Aridade	Descrição
⍒ (*grade down*)	diádico	Ranqueia o argumento descendentemente, de acordo com o alfabeto indicado

Unicode	TAB	APL
U+2352	V | <tab>	APL + Shift + 3

Reduções

- Em APL, um operador recebe um ou dois operandos (geralmente funções) como argumentos e retorna uma função (monádica ou diádica)
- O operador `/` (*reduce*) é monádico e retorna uma função ambivalente (que pode ser usada de forma monádica ou diádica)
- O nome diz respeito ao fato de que a função resultante reduz o *rank* do argumento em 1 unidade
- A redução de um vetor é direta: `F/a b c d e ...` equivale a `⊂ a F b F c F d F e F ...`, onde o resultado tem a mesma forma do argumento, exceto o último eixo

```
    +/ 2 3 5 7 11
28
    +/(2 3 5)(7 11 13)
9 14 18
```

Novo símbolo

Símbolo	Aridade	Descrição
/ (*reduce*)	operador	`F/v1 v2 v3 ... ≡ ⊂ v1 F v2 F v3 F ...`

Unicode	TAB	APL
U+002F	-	-

Características da redução

- A ordem de precedência das funções em APL (associativa à direita) afeta o comportamento da redução:

```
    -/ 2 3 5 7 11        ⍝ Soma alternada
8
    ⍟/2 3 5              ⍝ ⍟ é a função logaritmo
0.5508745883
```

- Em *arrays* com *rank* maior do que um, a redução se aplica sempre na última dimensão

```
    A ← 2 3 ⍴ ⍳6 ⋄ A
1 2 3
4 5 6
    ×/A
6 120
    ⍴ A ⋄ ⍴ ×/A
2 3
2
```

Novo símbolo

Símbolo	Aridade	Descrição
⍟ (*logarithm*)	diádico	Computa $\log_\alpha \omega$

Unicode	TAB	APL
U+235F	* O <tab>	APL + Shift + *

Novo símbolo

Símbolo	Aridade	Descrição
⍟ (*logarithm*)	monádico	Computa $\ln \omega$

Unicode	TAB	APL
U+235F	* O <tab>	APL + Shift + *

Redução nas demais dimensões

- O operador ⌿ (*reduce first*) produz uma redução em relação à primeira dimensão

```
    ×⌿A
4 10 18
```

- Para produzir uma redução em relação a k-ésima dimensão, use a notação `f/[k]`

```
    A ← 2 3 5 ⍴ ⍳9 ◇ A      ⍝ A tem duas matrizes com 5 linhas e 3 colunas cada
1 2 3 4 5
6 7 8 9 1
2 3 4 5 6

7 8 9 1 2
3 4 5 6 7
8 9 1 2 3
      +/[2] A               ⍝ A redução tem 2 linhas com colunas, cada elemento
 9 12 15 18 12              ⍝ correponde a soma das linhas das matrizes de A
18 21 15  9 12
```

- A notação `f/[1]` equivale a `f⌿`

Novo símbolo

Símbolo	Aridade	Descrição
⌿ (*reduce first*)	operador	Produz um redução em relação a primeira dimensão

Unicode	TAB	APL
U+233F	/ - <tab>	APL + ;

Reduções diádicas

- Na forma diádica, `L f/ R` é uma redução que usa uma janela de tamanho `L` em `R`
- Esta redução por janela não altera o *rank* do argumento

```
    2 +/ 2 3 5 7 11
5 8 12 18
    3 +/ 2 3 5 7 11
10 15 23
```

- Se `L` é negativo, a janela é invertida

```
    2 -/ 2 3 5 7 11
¯1 ¯2 ¯2 ¯4
    ¯2 -/ 2 3 5 7 11
1 2 2 4
```

Funções lógicas e reduções

- A função `all` retorna `1` se todos os booleanos do argumento são iguais a 1

```
      all ← ∧/
        all 1 0 0 1 0 1
0
```

- A função `any` retorna `1` se ao menos um booleano do argumento é igual a 1

```
      any ← ∨/
        any 1 0 0 1 0 1
1
```

- De fato, a função diádica `∧` computa o mínimo múltiplo comum de seus argumentos e equivale a conjunção lógica para valores booleanos
- A função diádica `∨` computa o maior divisor comum de seus argumentos, assumindo que $\gcd(0,0) = 0$, de forma que equivale à disjunção lógica para valores booleanos

Novo símbolo

Símbolo	Aridade	Descrição
∧ (*lcm*)	diádico	Computa o mínimo múltiplo comum entre **⍺** e **⍵**

Unicode	TAB	APL
U+2227	^ ^ <tab>	APL + O

Novo símbolo

Símbolo	Aridade	Descrição
∨ (*gcd*)	diádico	Computa o maior divisor comum entre α e ω

Unicode	TAB	APL
U+2228	v v <tab>	APL + 9

Fatorial

- É possível computar o fatorial de um inteiro positivo n de, no mínimo, três formas distintas em APL
- A primeira delas é recorrer a função monádica `!`, a segunda é implementar uma *dfn* recursiva e a terceira é usar uma redução
- A função **`cmpx`** do *workspace dfns* pode ser usada para comparar a performance destas três implementações

```
      ]CY 'dfns'
      f1←!
      f2←×/⍳
      f3←{⍵≤1:⍵⋄⍵×∇⍵-1}
      cmpx 'f1 100' 'f2 100' 'f3 100'
f1 100 → 1.8E¯7 |       0%
f2 100 → 3.0E¯7 |    +69%
f3 100 → 2.7E¯5 | +15339% ■■■
```

Varredura

- O operador `\` (*scan*) gera uma função monádica que age nos prefixos das últimas dimensões de seu parâmetro
- A expressão `f\a b c d ...` equivale a `(f/a) (f/a b) (f/a b c) ...`

```
    +\2 3 5 7 11        ⍝ somas acumuladas
2 5 10 17 28
    A ← 2 3 ⍴ ⍳6 ⋄ A
1 2 3
4 5 6
      ×\A
1  2   6
4 20 120
```

- O operador `⍀` gera uma varredura na primeira dimensão

Novo símbolo

Símbolo	Aridade	Descrição
\ (*scan*)	operador	Gera uma varredura na última dimensão

Unicode	TAB	APL
U+005C	-	-

Novo símbolo

Símbolo	Aridade	Descrição
⍀ (*scan first*)	operador	Gera uma varredura na primeira dimensão

Unicode	TAB	APL
U+2340	\ - <tab>	APL + .

Referências

1. APL Wiki. Defined function (traditional), acesso em 27/09/2021.
2. APL Wiki. Dfns workspace, acesso em 01/10/2021.
3. Dyalog. Try APL – Interactive lessons, acesso em 23/09/2021.
4. **IVERSON**, Kenneth E. *A Programming Language*, John Wiley and Sons, 1962.
5. Unicode Character Table. Página principal, acesso em 27/09/2021.
6. Xah Lee. Unicode APL Symbols, acesso em 23/09/2021.